

RS-232 Simulation Design

Documentation

Overview:

Our team has produced a program that can simulate the RS-232 communication protocol. This includes Non Return To Zero Encoding, Start and Stop Bits, Variable Data Length, and a Clock Drift simulation.

Main Function:

```
int main(){
//Initialize RS232 communication bus(integer)
//Internally we will represent negative voltage as 0 and positive voltage as 1
//Externally we will adjust to show the output
int rs232tx = 0;
int rs232rx = 0;
//Our two files
FILE* outputFile;
FILE* clockFile;
//allocate memory for the arguments
struct argData* args = malloc(sizeof(struct argData));
outputFile = fopen("output.txt", "w");
clockFile = fopen("idealClock.txt", "w");
//Simulate ideal clock
//check if it's null
if (clockFile != NULL){
    //Just run 50 cycles of a 9600bps baud rate clock.
    for(int i = 0; i < 50; i++){
        fprintf(clockFile, "%d %d\n", i%2, 52*i);
    }
}
//This way we can plot this against the actual packet and see the clock drift
fclose(clockFile);
//if the files not fucked(should really be checking this on the clock file too)
if(outputFile != NULL){
    //Add your data here! this is what we update every time we want to transmit
    stuff
    int data[3] = {0x68,0x69, -1};
    //Assigning all of our awesome variables!
    args->data = data;
```

```

args->tx = &rs232tx;
args->rx = &rs232rx;
args->file = outputFile;
//Need to dispatch a thread to do this
//need to set up a clock thread :D
pthread_t sender, clocker;
pthread_create(&clocker, NULL, clockThread, NULL);
pthread_create(&sender, NULL, sendData, args);
//We want to keep going after the sender is done
pthread_join(sender, NULL);
//we can just cancel the clock thread because we do not care about it :(
pthread_cancel(clocker);
//free the args
free(args);
//close the output file
fclose(outputFile);
}
else{
    printf("Unable to open file!\n");
}
return 0;
}

```

The main function starts by opening up the output file and an ideal reference clock file. It then generates the clock and closes the clock file. The data variable allows users to adjust the amount of data that the program will simulate transmission for. Since it is a typical standard for packets, only 8 bytes of data are allowed to be transmitted. This also sets up the variables handling the transmission and reception of the data.

From there, two threads are dispatched: one for the sender and one for the clock. The clock thread can be cancelled because we do not care about it terminating as we will discuss later. However the rest of the main function needs to wait for the send data function to finish. Afterwards, the args object is freed, the output file is closed, and the function returns zero.

Clock Implementation:

```

void* clockThread(void* args){
while(1){
    //Let's simulate clock drift
    int drift = (rand() % 13) - 6;
    usleep(104 + drift);
}
}

```

```

    //Need to make sure that our updated time in the sendData function is
    //reflective of the actual baud rate.
    baudRate = 104+drift;
    //lock da mutex
    pthread_mutex_lock(&mtx);
    //set clock ready to 1
    clock_ready = 1;
    //broadcast to signal the transmitting thread
    pthread_cond_broadcast(&clockTick);
    //Unlock the mutex
    pthread_mutex_unlock(&mtx);
}

```

This is a simple method of simulating clock drift. It is more akin to clock drift due to noise rather than hardware tolerances, but can still be used to test the integrity of communication systems. It simply adds +/- 6 uS to the time that it sleeps. Do note that the baudRate variable is global, and needs to be set globally for the clock drift is reflected in the output of the RS232 simulation. The variable clock_ready is set in this thread, which as shown later, is used to pause functionality in the send_data thread .

Send Data Implementation:

```

//Should support multiple threads
void* sendData(void* a)
{
    //Recasting to an argData pointer
    struct argData* args = (struct argData*)(a);
    //I did not know about fprintf at the time.
    char outText[50];
    //variable to keep track of time
    int time = 0;
    //Need to keep track of previous int to set the bus value
    int previousInt = 0;
    //Originally designed with these 4 as a single method running in the process.
    //Had to do this to make it multithreaded compatible.
    int* tx = args->tx;
    int* rx = args->rx;
    int* data = args->data;
    FILE* file = args->file;
    //Assuming maximum size of 8 bytes(pretty standard for a packet in digital
    //communications)
    for(int i = 0; i < 8; i++){
        if(data[i] == -1) break;
    }
}

```

```

//Start transmission
pthread_mutex_lock(&mtx);
*tx = 1;
//again, didn't know about fprintf at this time.
sprintf(outText, "%d %d\n", *tx, time);
fputs(outText, file);
//Wait for the clock to unlock
clock_ready = 0;
while(!clock_ready)
    pthread_cond_wait(&clockTick, &mtx);
pthread_mutex_unlock(&mtx);
//Update the time
time += baudRate;
for(int j = 0; j < 8; j++){
    //Get the first bit(LSB transmission)
    //Same as previous transmission: 1, different:0
    //Lock da mutex
    pthread_mutex_lock(&mtx);
    //actual logic for calculating what tx should be
    *tx = ((data[i] >> j) % 2 == previousInt) ? *tx : !(*tx);
    previousInt = (data[i] >> j) % 2;
    //print it to the file
    sprintf(outText, "%d %d\n", *tx, time);
    fputs(outText, file);
    //wait for the clock thread to signal this thread
    clock_ready=0;
    while(!clock_ready)
        pthread_cond_wait(&clockTick, &mtx);
    pthread_mutex_unlock(&mtx);
    //get back after it(update the time value)
    time += baudRate;
}
//Stop transmission
//Lock da mutex
pthread_mutex_lock(&mtx);
*tx = 0;
sprintf(outText, "%d %d\n", *tx, time);
fputs(outText, file);
//wait for the clock thread to signal this thread
clock_ready=0;
while(!clock_ready)
    pthread_cond_wait(&clockTick, &mtx);
pthread_mutex_unlock(&mtx);
//continue!

```

```

        time += baudRate;
    }
    return NULL;
}t

```

Need to admit, this is NOT my finest work 😬. The 50-character char buffer was used in tandem with `sprintf` and `fputs` to log the actual status of the transmission line along with the time. A sane person would simply use `fprintf`. The massive number of redundant assignments at the top were because during testing (and before multithreading support was added) I had all of those variables as parameters to the function. I had to convert them into a singular struct in post in order for it to work with `pthread_create`. The key functional patterns include:

- Setting `clock_ready` to zero and waiting for the clock thread to update it so it properly syncs up and the baud rate gets updated properly.
- Checking if the least significant bit is the same as the previous bit. If this is the case, the transmission line doesn't change, but if they differ, the bit needs to change. This is the implementation of the NRZI encoding.
- If `-1` is read, that means it's at the end of the list, and will stop transmitting.

Plotting documentation:

```

import sys
import numpy as np
import matplotlib.pyplot as plt

POS_V = 10.0 # RS-232 positive voltage
NEG_V = -10.0 # RS-232 negative voltage

def load_and_process(filename, rise_fraction=0.1):
    data = np.loadtxt(filename)

    levels = data[:, 0]
    times = data[:, 1]

    volts = np.where(levels > 0.5, POS_V, NEG_V)

    order = np.argsort(times)
    times = times[order]
    volts = volts[order]

    dt = np.median(np.diff(times)) if len(times) > 1 else 1.0

```

```
times_ext = np.concatenate([[times[0] - dt], times, [times[-1] + dt]])
volts_ext = np.concatenate([[NEG_V], volts, [NEG_V]])
```

```
t_new = []
v_new = []
```

```
for i in range(len(times_ext) - 1):
    t0 = times_ext[i]
    t1 = times_ext[i + 1]
    v0 = volts_ext[i]
    v1 = volts_ext[i + 1]
```

```
t_new.append(t0)
v_new.append(v0)
```

```
if v0 != v1:
    edge_dt = rise_fraction * (t1 - t0)
```

```
t_new.append(t1 - edge_dt)
v_new.append(v0)
```

```
t_new.append(t1)
v_new.append(v1)
```

```
t_new.append(times_ext[-1])
v_new.append(volts_ext[-1])
```

```
# convert time to ms
t_ms = np.array(t_new) / 1000.0
v_smooth = np.array(v_new)
```

```
return t_ms, v_smooth
```

```
def main():
    if len(sys.argv) < 2:
        print("Usage: python plot_rs232_multi.py file1.txt file2.txt ...")
        sys.exit(1)
```

```
plt.figure(figsize=(10, 4))
```

```
for i, fname in enumerate(sys.argv[1:]):
    t, v = load_and_process(fname)
```

```
# line styles
if i == 0:
    linestyle = "-"
```

```

elif i == 1:
    linestyle = "--" # SECOND FILE IS DASHED
else:
    linestyle = "-"

plt.plot(t, v, linestyle=linestyle, label=fname)

plt.xlabel("Time (ms)")
plt.ylabel("Voltage (V)")
plt.title("RS-232 Waveform")
plt.grid(True, linestyle=":", alpha=0.6)
plt.legend()
plt.tight_layout()
plt.show()

if __name__ == "__main__":
    main()

```

The plotting script reads the RS-232 output files generated by the C simulation, each containing pairs of logic levels (0 or 1) and corresponding timestamps. The script converts these logic levels into RS-232 voltage levels, mapping a logical '1' to +10 V and a logical '0' to -10 V, and also inserts idle periods of negative voltage before and after the transmitted data to reflect the true behavior of an RS-232 line at rest. To make the waveform visually realistic, the script adds short, sloped rise and fall segments at each transition instead of ideal vertical edges, producing a finite-slope approximation of physical signal behavior. The raw timestamps are then converted to milliseconds, and each processed signal is plotted on the same set of axes, with different line styles and colors used to distinguish multiple input files. The end result is a clean, time-aligned visualization that accurately reflects the transmitted data and the electrical characteristics of an RS-232 signal.

Example plot:

